

[TODO: title – current favourite: Communication-Aware DSE for IMC DNN Accelerators: Co-Optimizing Crossbar Packing, Tile Placement, and Adaptive 3D-NoC Selection]

[TODO: authors]

[TODO: affiliation]

Abstract—[TODO: abstract – write last]

I. INTRODUCTION

Deep Neural Networks (DNNs) have become the dominant workload class across computing. They underpin the vast majority of AI-based applications, and DNN inference has become infrastructure, running continuously from edge devices to warehouse-scale data centers [1]. The latency and energy budget of DNN accelerators is therefore a first-order design metric, and a primary axis of competition among chip designers and manufacturers. Weight movement is what drives this budget in conventional accelerators [2], [3]. In-Memory Computing (IMC) eliminates weight traffic by keeping weights in place and performing Multiply-Accumulate (MAC) operations locally. This alleviates the weight-movement bottleneck, improving performance while reducing the communication energy budget and launching a new generation of IMC DNN accelerators [4], [5], [6]. However, single array capacity is nowhere near occupying all weights for a full DNN, making tile architecture inevitable. For example one ResNet-50 block requires 736 crossbars (CBs) and a full network requires over ten thousand. Because ADC peripherals have a large area and power overhead, they are shared across the area of a tile. This limits the number of CBs that fit in a tile resulting in the computation being split among hundreds or even thousands of tiles connected through Network-on-Chip (NoC) [5].

The consequence is that IMC does not eliminate communication; it relocates it. What remains mobile is activation scatter traffic and the dominant partial-sum reduction traffic: with default packaging, roughly 82% of a block’s NoC bytes is reduction that converge on few accumulator tiles via the on-chip network. The accelerator’s bottleneck that governs its performance and energy becomes a communication, and the design focus shifts from being computation-centric to communication-centric. Communication on IMC DNN accelerators is not set by a single design choice, nor within a single design layer, but by a chain of dependent choices taken at different layers.

This paper presents a cross-layer design space exploration spanning three main layers—packing, mapping and routing—in which the design choices available at each layer are

evaluated both for their performance impact within that layer and for their impact on the layers downstream. A delay- and inference-time-aware design flow is proposed, and is shown to significantly reduce both average and tail delays, the latter measured as the p99 latency that governs inference time. The contributions are:

- a layer attribution of communication delay to packing and placement;
- the design flow itself, spanning the three layers;
- a mapping-layer design-time metric that maximises run-time gain at no communication-cost overhead;
- a bounding result that determines what design time can predict, motivating a future online adaptivity study.

Results show that the achieved improvement over the baseline reaches up to $11.6\times$ in mean delay and up to $18.7\times$ in p99 at matched load at the packing layer; up to 46% in mean delay and 67% in p99 with zero communication-cost overhead, at the mapping layer; and up to $1.85\times$ in mean delay and $2.39\times$ in p99 over a local heuristic at the routing and selection layer. Each gain was measured with the upstream layers already at their best, so downstream layers add to the upstream ones rather than trading against them.

II. BACKGROUND AND RELATED WORK

Wafer-scale integration shows what collapsing memory into compute costs at the largest scale yet built: Cerebras’ WSE-3 holds 44 GB on-wafer at 21 PB/s while provisioning 214 Pb/s of on-chip fabric — as reported, more aggregate bandwidth in the network than in the memory it serves [7]. Crossbar-based accelerators make the same collapse at tile granularity, so the interconnect rather than memory becomes the resource that mapping must be designed against [7].

III. FORMULATIONS AND METHODOLOGY

DNN traffic on a tiled IMC substrate is not the traffic classical NoC design assumes. It is a funnel rather than a permutation: roughly 82% of a block’s NoC bytes are partial-sum reduction converging on a handful of accumulator tiles, whose single ejection port is $18\text{--}148\times$ oversubscribed at true model rates. The binding constraint is therefore a port rather than a link, and it is invariant to placement and routing

— every placement reachable by ordinary means leaves the peak link load below that port floor. The traffic graph is itself a design variable, not an input: how much reduction is internalised on-tile is decided by the packing, which moves the port floor $5.8\times$ before any placement exists. It is also non-stationary — layer-sequential execution gives each flow a disjoint activity window inside one inference period, so peak and average diverge by $1.7\text{--}9.9\times$, separating feasibility from performance — and its congestion is transient and clustered at phase boundaries, with drain tails of $0.8\text{--}1.5\text{ k}$ cycles against phase windows of $5\text{--}23\text{ k}$, precisely the timescale an adaptive policy’s cost field must track. Finally, the endpoint that matters is the tail rather than the mean, because reduction is a barrier in which the accumulator waits for the slowest arriving partial sum; below saturation, throughput is uninformative — across configurations spanning 260% in delay, delivered throughput varied by 0.83% [REF].

This section defines the quantities that characterisation makes necessary, the design layers that set them, and the search that optimises them.

A. The three design layers

The top layer is *architecture and packing*. For a given hardware design it fixes how many crossbars share a tile (packing density, PD), and it decides the *layer-to-tile* mapping, i.e. how a DNN layer’s rows (reduce, r) and columns (scatter, s) are grouped across those crossbars (packing orientation, PO). Together, PD and PO fix how much of a layer’s traffic has to pass through a single tile port — a floor on delay that no later decision can lower. The second layer is *tile-to-node* placement, or *mapping*, which assigns NoC mesh nodes to those tiles and sets the peak single-link load across the NoC. The third is the *routing and selection* policy, which decides, at run time, how much of that load can be steered off that link.

The layers are searched in that order, and the search is nested rather than sequential: each PO induces its own traffic graph *and* its own placement space, so two orientations are comparable only when each is evaluated at its own best placement. Comparing a tuned placement of one orientation against an arbitrary placement of another confounds two layers.

B. Packing: PD, PO and the port floor

A packing (c, r, s) with $r \cdot s = c$ assigns the crossbars of a DNN layer to tiles: r groups crossbars along the reduction dimension, internalising partial-sum traffic on-tile, and s groups them along the scatter dimension, internalising input broadcast. It induces a *traffic graph* whose nodes are logical tiles and whose arcs are flows. Flow f carries rate λ_f flits/cycle from tile u_f to tile v_f and is active over the window $[t_f^{\text{on}}, t_f^{\text{off}})$ of the inference period T .

Because injection and ejection are independent channels, a tile’s two port rates bind separately. Over the elementary intervals $\mathcal{I}$ — the maximal windows on which the active flow

set is constant, cut at every t_f^{on} and t_f^{off} — the peak injection and ejection loads and the *port floor* are

$$\text{PIL} = \max_{I \in \mathcal{I}} \max_n \sum_{f: u_f=n, f \text{ active in } I} \lambda_f, \quad (1)$$

$$\text{PEL} = \max_{I \in \mathcal{I}} \max_n \sum_{f: v_f=n, f \text{ active in } I} \lambda_f, \quad (2)$$

$$\text{PF} = \max(\text{PIL}, \text{PEL}). \quad (3)$$

Equations (1)–(3) involve no placement variable, so PF is *placement-invariant*: remapping relabels which physical node is hot, never how hot it is. It is the quantity the packing layer alone controls, and $1/\text{PF}$ upper-bounds the load the design can sustain in a burst.

C. Placement: peak link load

A placement is an injective map π from tiles to mesh nodes. Under a routing function $\mathcal{R}$, let $\varphi_f(\ell, \pi) \in [0, 1]$ be the fraction of flow f ’s minimal $\mathcal{R}$ -admissible paths that traverse directed link ℓ , so that an even spread over admissible paths gives the offered link load $\Lambda_\ell = \sum_f \lambda_f \varphi_f(\ell, \pi)$, again maximised over $\mathcal{I}$. The placement layer is then characterised by

$$\text{PL} = \max_\ell \Lambda_\ell, \quad \text{CC} = \sum_f \lambda_f h(\pi(u_f), \pi(v_f)), \quad (4)$$

with h the Manhattan hop distance. PL is the only term a placement moves; CC is the hop-count energy proxy. Writing ℓ^* for the argmax link, the two layers meet at the *binding load*

$$\text{BIND} = \max(\text{PF}, \text{PL}), \quad (5)$$

which is the predictor the flow optimises: below PF the placement term is inactive and delay is flat in PL; above it PL binds and delay is steep.

D. Routing and selection

Routing is odd-even-balanced adaptive routing [8], a deadlock-free 3D turn model that admits, for most source–destination pairs, several minimal paths; $\mathcal{R}$ in (4) is this admissible set. Which admissible output a packet takes is the *selection* decision, and we compare two policies. **BL** is local: it picks the admissible output whose downstream input buffer has the most free slots, so its view of congestion extends one hop. **DP** is distributed and multi-hop: each router maintains, per destination, a congestion cost-to-go estimate obtained by relaxing its neighbours’ estimates over admissible turns, and selects the output minimising local occupancy plus that neighbour’s cost-to-go. DP therefore steers around congestion several hops ahead, at the cost of a field that must reconverge whenever the traffic pattern changes.

E. Proposed metrics

Selection can only divert load that has somewhere to go. For link ℓ , the share of its load carried by flows with an alternative is the *escapable fraction*

$$E(\ell) = 1 - \frac{1}{\Lambda_\ell} \sum_{f: \varphi_f(\ell)=1} \lambda_f, \quad (6)$$

the complement of the *forced* load, i.e. of the flows every one of whose admissible paths crosses ℓ . We write $E = E(\ell^*)$.

A single hot link, however, does not describe how the escape room is distributed across the links that actually congest. Let $\mathcal{L}_q$ be the top- q fraction of links ordered by Λ_ℓ , and let $E_q = \sum_{\ell \in \mathcal{L}_q} E(\ell) \Lambda_\ell / \sum_{\ell \in \mathcal{L}_q} \Lambda_\ell$ be the load-weighted escapable fraction of that tier. We propose the *escape slope*

$$ES = E_{20} - E_5, \quad (7)$$

the change in escape room between the top 5% and the top 20% of links. ES is computed offline from the traffic graph and the permutation alone, is invariant to the load scale, and — unlike E — is free at fixed PL and CC, which is what makes it usable as a refinement objective rather than a description.

Finally, because different placements saturate at different loads, capacity is measured per mapping: k^* is the injection scale at which a mapping reaches twice its *own* free-flow delay, interpolated from a load ladder. Mappings are compared at their own k^* , never at one global load.

F. The design flow and its search

The flow instantiates the layer order above: choose the orientation that minimises PF; place at minimum CC; refine that placement by maximising ES subject to $CC \leq CC_{\min}$ and $PL \leq PL(CC_{\min})$, which costs nothing at either upstream layer; minimise PL if further search is affordable; and run adaptive selection throughout.

All of (1)–(7) are evaluated from the traffic graph and the permutation alone, with no simulation. Placements are searched by a guarded hill-climb seeded from a pool of 1 000 random permutations: a step draws 60 candidate moves — a transposition of two occupied nodes, or, with probability 0.3, a relocation of one tile to an idle node — discards any candidate violating the active constraint, and accepts the best strictly improving one. Applying constraints inside the move filter rather than as a post-hoc rejection is what makes the guarded objectives above searchable directly. All searches are seeded and reproducible.

IV. RESULTS

A. Experimental Setup and Tools

All results are obtained with Noxim3D, a cycle-accurate SystemC network-on-chip simulator extended to three dimensions. The substrate is a $6 \times 6 \times 3$ mesh of 108 IMC tiles, each holding 128×128 crossbars; links and injection/ejection ports carry one 4-byte flit per 1 ns cycle, giving 4 GB/s per channel, with injection and ejection modelled as independent channels. Routing is odd-even-balanced adaptive routing [dahir2013oeb], on top of which we compare two selection policies: a local buffer-occupancy heuristic (BL) and a distributed multi-hop congestion cost-to-go field (DP) [9], whose cost field reconverges every 648 cycles and is clocked at $4 \times$ the network clock.

Workloads are ResNet-50, VGG-16 and DeiT-S, converted offline into statistical traffic tables whose rows carry per-pair injection rates and phase windows reproducing the layer-sequential structure of inference; one inference period is 38 536, 115 606 and 58 097 cycles respectively. Each measurement point is warmed up for 1 944 cycles and averaged over three seeds, with offered load swept by a scalar multiplier on all injection rates; unless stated otherwise, comparisons are made inside the knee window, defined as the load range in which mean BL latency stays below $30 \times$ its free-flow value. Placement search and all offline metrics (PF, PL, CC, ES) are computed in Python from the traffic tables and the permutation alone. The results below aggregate approximately 36 000 simulation runs.

Traffic is generated from statistical tables rather than dependency-driven traces. This bounds absolute latency accuracy, but not the comparisons the paper makes: every result is a ratio between placements measured under the same model, so a systematic bias cancels unless it reorders them. The model follows established practice for DNN NoC studies [10], and the phase windows preserve the layer-sequential structure that the temporal results depend on.

B. Metrics and operating point

For a packing (c, r, s) and a placement of its tiles onto the $6 \times 6 \times 3$ mesh, all predictors below are computed offline from the traffic table alone.

- **Port floor** $PF = \max(PIL, PEL)$: the peak per-node injection (PIL) or ejection (PEL) rate, in flits/cycle. Both terms are fixed by the flow graph, so PF is *placement-invariant*: remapping relabels which node is hot, never how hot it is (verified identical to six decimal places across random permutations).
- **Peak link load** PL: the maximum offered rate on any router-to-router channel when each flow spreads evenly over its admissible minimal paths under the routing function. PL is the only term a placement moves.
- **Binding load** $BIND = \max(PF, PL)$.
- **Escapable fraction** $E = 1 - PL_f / PL$ at the peak link, where PL_f counts only flows for which *every* admissible path crosses that link. E is the share of the hot link's load that a selection policy could, in principle, divert.
- **Per-mapping capacity** k^* : the injection scale at which a mapping reaches twice its own free-flow delay, interpolated from a load ladder. Mappings are compared at their own k^* , never at one global load [TODO: cite the k^* method note].

Load is reported as the fraction $PF \cdot k$ of the binding port's capacity; the three workloads all knee at 44–48% of the dominant reduction sink's ejection bandwidth, so this normalization is workload-independent.

C. Packing sets the floor, and minimum traffic does not minimize it

At fixed density c , the orientation (r, s) decides how much of the partial-sum reduction is internalised on-tile and how

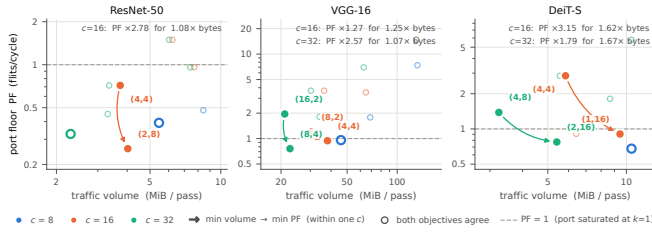


Fig. 1. The packing design space: traffic volume against port floor PF, all feasible (c, r, s) points, argmins taken *within* each density c . Arrows run from the minimum-volume orientation to the minimum-PF orientation at the same c ; rings mark cells where the two objectives agree. This figure argues about the floor; that the floor determines delay is established in Fig. 2.

much reaches the NoC, and thereby sets PF: across ResNet-50’s eleven feasible (c, r, s) points PF spans $0.258\text{--}1.494$ flits/cycle ($5.8\times$) before a single placement decision is made. Placement, by contrast, moves only PL (span $7.5\times$ within one packing). The two terms of BIND therefore belong to different design layers — PF to packing, PL to placement — and the rest of this section establishes which layer holds control in which regime.

Fig. 1 plots every feasible packing of the three workloads in the (traffic volume, PF) plane. The classical pre-NoC objective — minimize communication volume — and the architecture-aware objective — minimize PF — select *different* orientations in five of the eight (*workload, c*) cells where more than one orientation exists. The disagreement is not marginal: at $c=16$, ResNet-50’s minimum-volume orientation $(4, 4)$ carries a $2.78\times$ higher floor than $(2, 8)$, which costs only 8% more traffic; VGG-16 at $c=32$ shows the same pattern ($2.57\times$ the floor, 7% traffic). Where the trade is expensive the figure says so: DeiT-S buys its $1.8\text{--}3.2\times$ floor reductions with $1.6\times$ the traffic. Minimum bytes selects the burstiest packings; it is the wrong objective whenever the floor, not the volume, binds — which Sec. IV-D shows is exactly the regime that matters.

D. The regime boundary: delay is flat below the floor and steep above

Fig. 2 plots mean delay against PL for 24 placements of one packing spanning $PL = 0.127\text{--}0.490$ ($3.9\times$) at two loads, with path variety pinned so PL is the only moving predictor. Delay does not respond to PL until PL crosses PF, and responds sharply after: a hinge regression at the floor yields $R^2 = 0.735$ (vs. 0.404 linear), with a below-floor slope indistinguishable from zero ($t = -0.16$) and an above-floor slope of $+655$ cycles per unit load ($t = +5.13$, $p < 10^{-4}$). The hinge replicates at a second load level ($R^2 = 0.705$; slope ratio $64\text{--}72\times$ across k) [TODO: regenerate fig with the E1 grid; cite exact fig file].

Below the floor, the packing orientation is the determining layer. With BIND = PF, delay tracks the floor itself [TODO: E5: PF varied through (r, s) at matched k — currently varied through k only; run E5 before submission], and placement is inert: seventeen placements spanning $3.07\times$ in PL move mean delay $1.15\times$ (n.s.). We stress the construc-

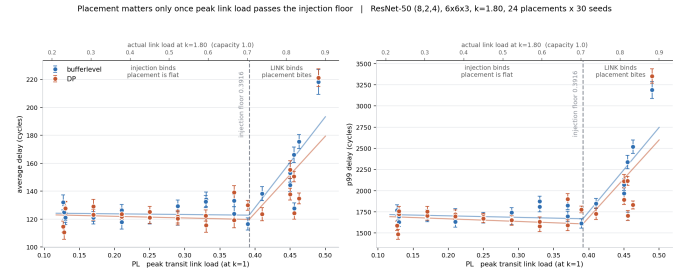


Fig. 2. Mean delay against peak link load PL under local (buffer-level) selection, PF marked. Below the floor a $3.07\times$ spread in PL moves delay $1.15\text{--}1.18\times$ (n.s.); above it a $1.20\times$ spread moves delay $1.71\text{--}1.85\times$. [TODO: replace with E1-grid version, BL only]

tive reading: *verifying $PL < PF$ frees the entire mapping budget* for objectives that are not latency — thermal spreading, energy, load balance — at no delay or p99 penalty. On DeiT-S this licence replicates end-to-end: at two packings held below their floors, a placement searched for escapable fraction plus a free choice of selection policy buys $0.99\text{--}1.11\times$ over the classical minimum-hop mapping with local selection ($1,344$ runs; indistinguishable from parity at most loads).

The licence carries one condition: *margin*. Among eight placements all below the floor, capacity k^* still varies $1.28\times$ and correlates with proximity to the boundary ($r(PL/PF, k^*) = -0.70$); at mid-load the two placements nearest the floor degrade $13.6\times$ earlier than the rest. The budget is free at $PL \lesssim 0.6 PF$, not asymptotically up to the boundary.

Above the floor, control passes to placement. Across the 24-placement grid of Sec. IV-F, PL predicts per-mapping capacity at $r(PL, k^*) = -0.60$ ($p = 0.002$), and capacity spans $1.50\times$ across placements of the *same* packing. This is also the first simulator validation of the offline PL model ($R^2 = 0.37$): a moderate capacity predictor, no more [TODO: VGG replication: $r(PL, k^*) = -0.72$, BL arm; DP arm and DeiT-S pending (E7)].

E. Selection pays only above the floor, and only where load can escape

Table I compares the distributed cost-to-go policy (DP) against the local buffer-level heuristic (BL) on identical placements, split by regime. Below the floor the policies are statistically indistinguishable — no routing decision reaches a flit that has not entered the network — while above it DP removes $7\text{--}10\%$ of mean delay. The gain is not universal: it requires escapable load. Pooled across five workload/packing arms ($n=71$ placements), DP’s capacity gain tracks the escapable fraction E on ejection-bound packings ($r = +0.59$, $p = 0.0002$) and vanishes on injection-bound ones ($r = +0.07$, n.s.), where the binding port is upstream of every routing decision.

TABLE I
DP VS. BL MEAN DELAY ON IDENTICAL PLACEMENTS, BY REGIME (24
PLACEMENTS, $n=30$ SEEDS).

regime	$k = 1.60$		$k = 1.80$	
	Δ	p	Δ	p
below floor ($n=17$)	-0.0%	0.98	-1.1%	0.37
above floor ($n=7$)	-10.0%	0.0005	-6.7%	0.008

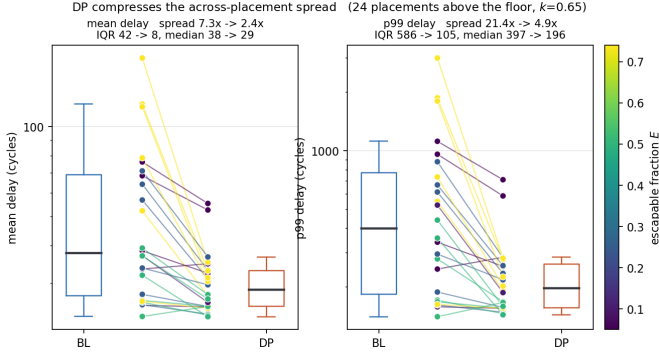


Fig. 3. Delay across placements of equal peak link load, BL vs. DP, above the floor ($k = 0.65$). DP compresses the spread across placements rather than shifting its floor. [TODO: final styling pass; confirm k in caption]

F. Adaptive selection returns the mapping freedom that congestion took

The headline effect of DP is not its mean — it is its *variance across placements*. Fig. 3 compares the two policies over placements of equal PL above the floor. DP does not make the best placements faster (BL matches or beats it there); it removes the tail: across 21 equal-PL placements the delay spread contracts from $1.59\times$ to $1.45\times$ ($CV\ 0.145 \rightarrow 0.101$), and in a population containing BL-collapse cases from $64\times$ to $8.6\times$. On the capacity axis the same grid gives DP a mean $+10.3\%$ k^* gain ($p = 2.3 \times 10^{-4}$, positive on 18/24), sized by E : a two-predictor model gain $\sim E + PL$ reaches adjusted $R^2 = 0.55$, and every placement where DP loses is low- E .

The design consequence mirrors Sec. IV-D: once a PL level is fixed, the choice *among* equal-PL placements matters far less under DP than under BL — adaptive selection extends the below-floor mapping licence into the congested regime. What it does not do is substitute for the placement decision itself: the k^* spread across PL levels survives under both policies.

G. Scope and threats to validity

PL, PL_f and E are offline path-enumeration models; only PL has been validated against the simulator, and only as a moderate capacity predictor ($R^2 = 0.37$). The even-spreading assumption behind PL models an ideal balancer, which BL is not — BL underperforms its own PL estimate in proportion to E . All results use one $6\times 6\times 3$ mesh; mesh-size generality is out of scope. Partial sums are carried at INT32, a convention that *overstates* the reduction traffic the contribution exploits; the placement rankings survive at 16-bit for ResNet and VGG. Cross-workload replication is complete for the hinge and the

licence, partial for the selection results [TODO: close with E7 + VGG DP arm, then finalize].

V. CONCLUSION

[TODO: conclusion + the three directions (temporal DP, thermal-aware mapping, router design for reduction sinks)]

REFERENCES

- [1] e. a. Jouppi, “In-datacenter performance analysis of a tensor processing unit,” vol. 45, no. 2. New York, NY, USA: Association for Computing Machinery, Jun. 2017, p. 1–12. [Online]. Available: <https://doi.org/10.1145/3140659.3080246>
- [2] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.
- [3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [4] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory devices and applications for in-memory computing,” *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, “Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [6] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, “Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27–39.
- [7] T. Miyajima and L. Fukuoka, “Benchmarking the cerebras wafer scale engine-2 architecture,” ser. SC Workshops ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 818–822. [Online]. Available: <https://doi.org/10.1145/3731599.3767438>
- [8] N. Dahir, T. Mak, F. Xia, and A. Yakovlev, “Highly adaptive and deadlock-free routing for three-dimensional networks-on-chip,” *IET Computers & Digital Techniques*, vol. 7, no. 6, pp. 255–263, 2013, vERIFY author list and DOI against the published paper.
- [9] T. Mak, P. Y. Cheung, W. Luk, and K. P. Lam, “A dp-network for optimal dynamic routing in network-on-chip,” in *Proceedings of the 7th IEEE/ACM International Conference on Hardware/Software Codesign and System Synthesis*, ser. CODES+ISSS ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 119–128. [Online]. Available: <https://doi.org/10.1145/1629435.1629452>
- [10] G. Krishnan, S. K. Mandal, C. Chakrabarti, J.-s. Seo, U. Y. Ogras, and Y. Cao, “Impact of on-chip interconnect on in-memory acceleration of deep neural networks,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 18, no. 2, Apr. 2022, online first 31 Dec 2021; both 2021 and 2022 appear in listings. Title/authors/venue/volume/DOI verified; VERIFY the per-pair injection-rate traffic model before citing it as precedent.